ChatGPT Conversation Documentation (Sebastian Cadiz)

CISC106-084 11/13/2026

Material Strength Explorer | Me ChatGPT

- I made the code smaller to reduce the amount of pages.

Hey Chat, I'm currently working on a project, and I've hit a stuck point. My application is going well so far, i've been working on it for a while. It is about the Material Strength Explorer that we brainstormed about the other day. I need your assistance with making a graph using Matplotlib and enhancing my application overall

I've received feedback from my classmates about some problems with my application. I want to improve the overall design and functionality while keeping the project aligned with my course requirements, also, here is my current code:

```
from drafter import *

from dataclasses import dataclass

import numpy as np

from scipy.spatial.distance import cdist



#Dataclasses

@dataclass

class Material:

  """Stores the physical properties of one metal or alloy."""

  name: str

  density: float

  youngs_modulus: float

  yield_strength: float



@dataclass

class State:

  """Stores all information that needs to persist across pages."""

  materials: list[Material]

  density_input: float

  modulus_input: float

  yield_input: float

  matched_materials: list[Material]

  comparison_scores: dict[str, float]

  status_message: str
```

#Material Database

def create_material_database() -> list[Material]:

    """Creates the reference database containing 15 materials."""

    return [

        Material("Aluminum 6061-T6", 2.70, 69.0, 276),

        Material("Aluminum 7075-T6", 2.81, 71.7, 503),

        Material("Aluminum 2024-T3", 2.78, 73.1, 345),

        Material("AISI 1018 Steel", 7.87, 205, 370),

        Material("AISI 1045 Steel", 7.87, 205, 310),

        Material("AISI 4140 Steel", 7.85, 205, 655),

        Material("A36 Structural Steel", 7.85, 200, 250),

        Material("Stainless Steel 304", 8.00, 193, 215),

        Material("Stainless Steel 316", 8.00, 193, 205),

        Material("Copper C110", 8.89, 117, 70),

        Material("Brass C260", 8.53, 110, 95),

        Material("Bearing Bronze", 8.80, 110, 125),

        Material("Titanium Ti-6Al-4V", 4.43, 113.8, 880),

        Material("Magnesium AZ31B", 1.77, 45, 200),

        Material("Nickel 200", 8.89, 204, 148)

    ]

#Helper Functions

def calculate_matches(state: State) -> None:

    """Compares user measurements against every database material."""

    user_values = np.array([[

        state.density_input,

        state.modulus_input,

        state.yield_input

    ]])

    material_values = []

    for material in state.materials:

        material_values.append([

            material.density,

            material.youngs_modulus,

            material.yield_strength

        ])

    material_values = np.array(material_values)

    scales = np.ptp(material_values, axis=0)

    distances = cdist(

        user_values / scales,

        material_values / scales,

```
        metric="euclidean"
    )[0]

    state.comparison_scores = {}
    for index in range(len(state.materials)):
        name = state.materials[index].name
        state.comparison_scores[name] = float(distances[index])

    closest_indices = np.argsort(distances)[:2]
    state.matched_materials = []
    for index in closest_indices:
        state.matched_materials.append(state.materials[index])

    if distances[closest_indices[0]] < 0.10:
        state.status_message = "A close material match was found!"
    else:
        state.status_message = (
            "The closest match may not be reliable. "
            "Check the comparison values."
        )

#Home Page
@route
def index(state: State) -> Page:
    """Displays the homepage and navigation options."""
    return Page(state, [
        Header("Material Strength Explorer"),
        "Identify an unknown metal using its physical properties.",
        HorizontalRule(),
        "Enter three measurements to find the closest "
        "matching metal or alloy in our database.",
        Button("Identify a Material", "input_page"),
        Button("Browse Material Database", "database_page"),
        Button("About This Project", "about_page")
    ])

#Input Page
@route
def input_page(state: State) -> Page:
    """Displays the form for entering material properties."""
    return Page(state, [
        Header("Material Identification"),
        "Enter the measured properties of your unknown material.",
        HorizontalRule(),
        "Density (g/cm^3):",
```

```
        TextBox("density", 2.70, kind="number"),
        "\n",
        "Young's Modulus (GPa):",
        TextBox("modulus", 69.0, kind="number"),
        "\n",
        "Yield Strength (MPa):",
        TextBox("yield_strength", 276, kind="number"),
        "\n",
        Button("Identify Material", "submit_material_properties"),
        Button("Home", "index")
    ])

#Submit and Process Input
@route
def submit_material_properties(
    state: State,
    density: float,
    modulus: float,
    yield_strength: float
) -> Page:
    """Validate user measurements and calculate material matches."""

    if density <= 0 or modulus <= 0 or yield_strength <= 0:
        state.status_message = "All measurements must be positive."
        return input_page(state)

    if density > 25 or modulus > 600 or yield_strength > 3000:
        state.status_message = "One or more measurements are unrealistic."
        return input_page(state)

    state.density_input = density
    state.modulus_input = modulus
    state.yield_input = yield_strength

    calculate_matches(state)
    return results_page(state)

#Results Page
@route
def results_page(state: State) -> Page:
    """Displays the closest matches and comparison information."""
    if len(state.matched_materials) == 0:
        return Page(state, [
            Header("No Results Yet"),
            "Please enter material measurements first.",
```

```
        Button("Enter Measurements", "input_page")
    ])

    best_match = state.matched_materials[0]
    second_match = state.matched_materials[1]

    return Page(state, [
        Header("Material Identification Results"),
        HorizontalRule(),
        "Your measurements:",
        "Density: " + str(state.density_input) + " g/cm^3",
        "Young's Modulus: " + str(state.modulus_input) + " GPa",
        "Yield Strength: " + str(state.yield_input) + " MPa",
        HorizontalRule(),
        Header("Best Match", 2),
        best_match.name,
        state.status_message,
        Header("Second Closest Match", 2),
        second_match.name,
        HorizontalRule(),
        "Comparison scores (smaller is better):",
        "Best match: " + str(
            state.comparison_scores[best_match.name]
        ),
        "Second match: " + str(
            state.comparison_scores[second_match.name]
        ),
        Button("View Full Database", "database_page"),
        Button("Try Again", "reset_inputs"),
        Button("Home", "index")
    ])

#Database Page
@route
def database_page(state: State) -> Page:
    """Displays all materials in the reference database."""
    content = [
        Header("Material Database"),
        "Our database currently contains 15 reference materials.",
        HorizontalRule()
    ]
    for material in state.materials:
        content.append(material.name)
        content.append(
            "Density: " + str(material.density) + " g/cm^3"
```

```
        )
        content.append(
            "Young's Modulus: " +
            str(material.youngs_modulus) + " GPa"
        )
        content.append(
            "Yield Strength: " +
            str(material.yield_strength) + " MPa"
        )
        content.append(HorizontalRule())
    content.append("Select a material to view its details:")
    content.append(
        SelectBox(
            "material_name",
            [material.name for material in state.materials]
        )
    )
    content.append(Button("View Material", "material_detail_page"))
    content.append(Button("Home", "index"))
    return Page(state, content)


#Material Details Page
@route
def material_detail_page(
    state: State,
    material_name: str
) -> Page:
    """Displays the properties of one selected material."""
    for material in state.materials:
        if material.name == material_name:
            return Page(state, [
                Header(material.name),
                HorizontalRule(),
                "Density: " + str(material.density) + " g/cm^3",
                "Young's Modulus: " +
                str(material.youngs_modulus) + " GPa",
                "Yield Strength: " +
                str(material.yield_strength) + " MPa",

                Button("Back to Database", "database_page"),
                Button("Home", "index")
            ])
    return Page(state, [
        "Material not found.",
        Button("Back to Database", "database_page")
```

```
    ])


#About Page

@route

def about_page(state: State) -> Page:

    """Explain the purpose and limitations of the application."""


    return Page(state, [

        Header("About Material Strength Explorer"),

        HorizontalRule(),

        "This application identifies unknown metals and alloys "

        "by comparing three material properties.",

        "The three properties are density, Young's modulus, "

        "and yield strength.",

        "NumPy organizes and processes the numerical data, "

        "while SciPy calculates the distance between the "

        "user's measurements and the reference materials.",

        "The application only provides estimated matches. "

        "Materials with similar properties may be difficult "

        "to distinguish using these three measurements alone.",

        Button("Home", "index"),

        Button("Identify a Material", "input_page")

    ])


#Reset

@route

def reset_inputs(state: State) -> Page:

    """Clears previous measurements and matching results."""

    state.density_input = 0.0

    state.modulus_input = 0.0

    state.yield_input = 0.0

    state.matched_materials = []

    state.comparison_scores = {}

    state.status_message = ""

    return input_page(state)


#Drafter Start

start_server(State(

    materials=create_material_database(),

    density_input=0.0,

    modulus_input=0.0,

    yield_input=0.0,

    matched_materials=[],

    comparison_scores={},

    status_message=""
```

Sure! I can help refine your code and incorporate a Matplotlib graph. Since your application compares material properties, a stress-strain graph would be a useful addition.

We can use Young's modulus and yield strength to calculate and display the linear-elastic portion of a stress-strain curve. The graph will help users understand how a material responds to applied stress.

I'll also review the existing application for opportunities to improve the navigation, input handling, and database presentation without unnecessarily complicating your code.

I want to make sure the graph actually makes sense with the information I have in my database

Your database contains Young's modulus and yield strength, so we can calculate the linear-elastic portion of a stress-strain curve.

The relationship is:

Stress = Young's modulus × Strain

We can calculate the strain at the yield point by dividing yield strength by Young's modulus.

One limitation is that your database does not contain enough information to accurately model what happens after yielding. Therefore, the graph should only represent the elastic region rather than inventing plastic deformation or fracture behavior.

I also want to remove the property values from the database listing. I think it would be better if the database only showed the material names, and then users could select a material to view its individual properties

That is a good improvement. Displaying every property directly on the database page makes it crowded and reduces the usefulness of the individual material details page.

We can change the database page so that it displays only the material names. Users can select a material and click a button to open its details page.

The details page will display the density, Young's modulus, and yield strength of the selected material.

Can we make the input fields initially blank instead of having values like 2.7, 69, and 276 already filled in

Yes. The input fields should start blank so users understand that they need to enter their own measurements.

We can use empty default values for the input fields. We should also make sure that previously entered values remain visible if the user submits invalid measurements.

What happens if someone enters something like 999? I don't want the program to reset everything to the default values

The program should not silently replace an invalid measurement with a default value.

Instead, it should display an error message explaining why the measurement is invalid. We can also preserve the user's input so they can correct it without having to reenter everything.

We should validate that:

- All three measurements are numeric.
- All measurements are positive.
- The measurements are within reasonable limits.

For example, a density of 999 would be rejected as unrealistic, and the user would receive an appropriate error message.

What is the purpose of HorizontalRule() Is it necessary

HorizontalRule() creates a horizontal divider on the page. It separates sections and makes the website easier to read.

It is optional. It does not affect the calculations or functionality of your application, so you can keep it for visual organization or remove it if you prefer a simpler layout.

I also need to add unit tests to the project

We can add unit tests to verify that important functions work correctly.

For example, we can test whether the database contains the correct number of materials, whether the matching algorithm identifies a known material correctly, and whether resetting the application clears the previous measurements and results.

These tests should run before start_server() so that potential errors are caught before the website launches.

Can't we do the unit tests using from bakery import assert_equal

Yes! We can absolutely use `from bakery import assert_equal`.

Since your course uses the Bakery library, that is a good approach for your project. We can write tests using assert_equal rather than relying on ordinary Python assert statements.

For example:

None

```
def test_create_material_database():
    materials = create_material_database()

    assert_equal(len(materials), 15)
    assert_equal(materials[0].name, "Aluminum 6061-T6")
```

We can also test the matching algorithm and reset functionality.

Okay, but I need you to input the assert_equal tests with the whole code later so I can actually copy it into Thonny and run it.

Absolutely. I'll provide the complete program with the Bakery import and unit tests already incorporated.

The tests will be placed before the website startup code. This way, they execute first, and the website starts only after the tests pass.

The tests will cover:

1. Creating the material database.
2. Checking the database's material properties.
3. Finding the correct closest material.
4. Returning two matching materials.
5. Resetting the application's State fields.

The expected output in the Thonny Shell will be:

None

```
All 3 unit tests passed!
```

Afterward, Drafter should launch the website.

Okay, you can now paste the polished code and also document this conversation as a transcript, thanks

Understood. The documentation will follow the chronological order of our discussion.

Add this code to your current version:

None

```
import matplotlib.pyplot as plt



def calculate_stress_strain(
    material: Material
) -> tuple[np.ndarray, np.ndarray]:
    """Calculate a simplified linear-elastic stress-strain curve."""
    # Convert Young's modulus from GPa to MPa.
    modulus_mpa = material.youngs_modulus * 1000
    # Calculate the strain at which the material reaches yield.
    yield_strain = material.yield_strength / modulus_mpa
    # Generate 100 strain values from zero to the yield point.
    strain_values = np.linspace(0, yield_strain, 100)

    # Calculate stress using the linear-elastic equation.
    stress_values = modulus_mpa * strain_values
    return strain_values, stress_values


def plot_stress_strain(material: Material) -> None:
    """Create a graph showing the material's elastic behavior."""

    strain_values, stress_values = calculate_stress_strain(material)

    # Convert strain to a percentage for easier reading.
    strain_percent = strain_values * 100
    yield_strain_percent = strain_percent[-1]

    plt.figure(figsize=(8, 5))

    plt.plot(
        strain_percent,
        stress_values,
        label="Elastic Region"
    )
    plt.scatter(
        [yield_strain_percent],
        [material.yield_strength],
        label="Yield Point"
    )
```

```
    plt.xlabel("Strain (%)")
    plt.ylabel("Stress (MPa)")
    plt.title(material.name + " Stress-Strain Curve")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()


#Input Page
#route
def input_page(state: State) -> Page:
    """Displays the form for entering material properties."""
    # Empty boxes appear when no measurements have been entered.
    density_default = ""
    modulus_default = ""
    yield_default = ""
    # Preserve previous numeric entries when validation fails.
    if state.density_input != 0:
        density_default = str(state.density_input)
    if state.modulus_input != 0:
        modulus_default = str(state.modulus_input)


    if state.yield_input != 0:
        yield_default = str(state.yield_input)
    content = [
        Header("Material Identification"),
        "Enter the measured properties of your unknown material.",
        HorizontalRule()
    ]
    # Display an error message when one exists.
    if state.status_message != "":
        content.append(state.status_message)
        content.append("\n")
    content.extend([
        "Density (g/cm^3):",
        TextBox("density", density_default, kind="number"),
        "\n",
        "Young's Modulus (GPa):",
        TextBox("modulus", modulus_default, kind="number"),
        "\n",
        "Yield Strength (MPa):",
```

```
        TextBox("yield_strength", yield_default, kind="number"),
        "\n",
        Button("Identify Material", "submit_material_properties"),
        Button("Home", "index")
    ])
    return Page(state, content)


#Submit and Process Input
#route
def submit_material_properties(
    state: State,
    density: str,
    modulus: str,
    yield_strength: str
) -> Page:
    """Validate user measurements and calculate material matches."""
    # Convert the three inputs into numbers.
    try:
        density_value = float(density)
        modulus_value = float(modulus)
        yield_value = float(yield_strength)
    except ValueError:
        state.status_message = (
            "Error: Please enter a number in all three fields."
        )
        return input_page(state)
    # Save the entered measurements.
    state.density_input = density_value
    state.modulus_input = modulus_value
    state.yield_input = yield_value
    # Check whether the inputs are finite numbers.
    if (not np.isfinite(density_value)
            or not np.isfinite(modulus_value)
            or not np.isfinite(yield_value)):
        state.status_message = (
            "Error: Measurements must be finite numbers."
        )
        return input_page(state)
    # Check whether all measurements are positive.
    if (density_value <= 0
```

```
            or modulus_value <= 0
            or yield_value <= 0):
        state.status_message = (
            "Error: All measurements must be positive."
        )
        return input_page(state)
    # Check whether the measurements are within reasonable bounds.
    if (density_value > 25
            or modulus_value > 600
            or yield_value > 3000):
        state.status_message = (
            "Error: One or more measurements are outside "
            "the reasonable bounds for this application."
        )
        return input_page(state)
    state.status_message = ""
    calculate_matches(state)
    return results_page(state)


#Results Page
@route
def results_page(state: State) -> Page:
    """Displays the closest matches and comparison information."""
    if len(state.matched_materials) == 0:
        return Page(state, [
            Header("No Results Yet"),
            "Please enter material measurements first.",
            Button("Enter Measurements", "input_page")
        ])
    best_match = state.matched_materials[0]
    second_match = state.matched_materials[1]
    # Generate the graph for the best matching material.
    plot_stress_strain(best_match)
    return Page(state, [
        Header("Material Identification Results"),
        HorizontalRule(),
        "Your measurements:",
        "Density: " + str(state.density_input) + " g/cm**3",
        "Young's Modulus: " + str(state.modulus_input) + " GPa",
        "Yield Strength: " + str(state.yield_input) + " MPa",
```

```
        HorizontalRule(),
        Header("Best Match", 2),
        best_match.name,
        state.status_message,
        Header("Second Closest Match", 2),
        second_match.name,
        HorizontalRule(),

        "Comparison scores (smaller is better):",
        "Best match: " + str(
          state.comparison_scores[best_match.name]
        ),
        "Second match: " + str(
          state.comparison_scores[second_match.name]
        ),
        HorizontalRule(),
        Header("Stress-Strain Graph", 2),
        "The graph shows the linear-elastic region up to the "
        "material's yield point.",
        MatPlotLibPlot(),
        Button("View Full Database", "database_page"),
        Button("Try Again", "reset_inputs"),
        Button("Home", "index")
    ])


#Database Page
@route
def database_page(state: State) -> Page:
    """Displays the names of all materials in the database."""
    content = [
        Header("Material Database"),
        "Our database currently contains 15 reference materials.",
        "Select a material to view its individual properties.",
        HorizontalRule()
    ]
    # Only display material names to avoid repeating the details page.
    for material in state.materials:
        content.append(material.name)
    content.extend([
        HorizontalRule(),
```

```
        "Select a material:",
        SelectBox(
            "material_name",
            [material.name for material in state.materials]
        ),
        Button("View Material", "material_detail_page"),
        Button("Sort Alphabetically", "sort_database"),
        Button("Home", "index")
    ])
    return Page(state, content)


#Sort Database
@route
def sort_database(state: State) -> Page:
    """Sort the material database alphabetically."""
    state.materials.sort(key=lambda material: material.name.lower())
    return database_page(state)


#Unit Tests
.

#Drafter Start
start_server(State(
    materials=create_material_database(),
    density_input=0.0,
    modulus_input=0.0,
    yield_input=0.0,
    matched_materials=[],
    comparison_scores={},
    status_message=""
))
```